Security and Policy-Guided UAV Systems

Performance Benchmarking of Attestation Schemes

M.Tech Thesis Presentation

Presentation By - Aastha Jain [JCS242802]
Guided By - Prof. Vireshwar Kumar

March 7, 2026

Outline

1. Motivation & Introduction
2. Background
3. Threat Model
4. Integrity Verification & Attestation Schemes
5. Why Benchmarking?
6. Implementation
7. Challenges
8. Results & Analysis
9. Conclusion & Future Work

Why Do We Care About UAV Security?

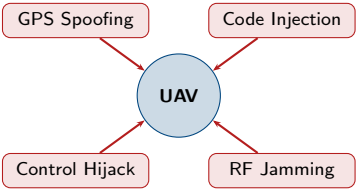
UAVs Are Everywhere

Unmanned Aerial Vehicles are now used in:

- Military reconnaissance & surveillance
- Package delivery (Amazon, Zomato)
- Precision agriculture
- Search & rescue operations
- Infrastructure inspection

The Problem

UAVs run **safety-critical** software **autonomously** with little human supervision. A compromised drone can crash, be hijacked, or cause physical harm. Existing flight-control stacks have **no runtime integrity guarantees**.



Attack surface of a modern UAV

What Is Our Focus?

Can we verify that a UAV's flight-control software is executing exactly as intended — at runtime?

[FIND] Integrity Verification

We want to **detect** any deviation from the expected execution path — even if the attacker has code-level access.

[CHART] Performance Benchmarking

Attestation adds overhead. We must measure **how much** to assess real-world deployability on embedded hardware.

[SCALE] Schemes Compared

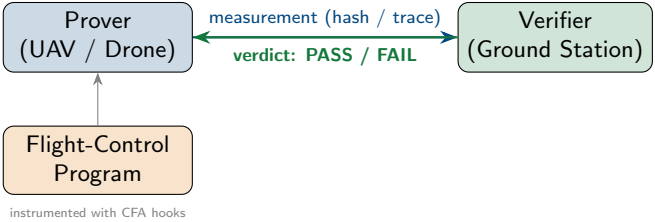
We compare three attestation schemes:

- Branch Counter
- OAT
- C-FLAT

Background: Control-Flow Attestation (CFA) — The Big Picture

What is Attestation?

Attestation is the process by which a *prover* (the UAV) convinces a *verifier* (ground station) that its software is running correctly and has not been tampered with.



Control-Flow Attestation specifically verifies that the program followed a *legitimate control-flow path* — it cannot be fooled by code-reuse attacks (ROP/JOP) that only jump between valid code fragments.

Background: ARM TrustZone

What is TrustZone?

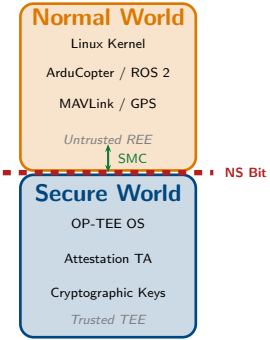
ARM TrustZone is a **hardware security extension** that divides the processor into two isolated worlds:

- **Normal World (REE):** Runs Linux, ArduCopter, ROS 2 — untrusted application code.
- **Secure World (TEE):** Runs OP-TEE OS — stores secrets, performs attestation; isolated from Normal World by hardware.

A hardware `NS` bit enforces the separation.

Why Does CFA Need TrustZone?

An attacker controlling Normal World can forge a software attestation log. Only a **hardware root-of-trust** (TrustZone TEE) produces unforgeable measurements.



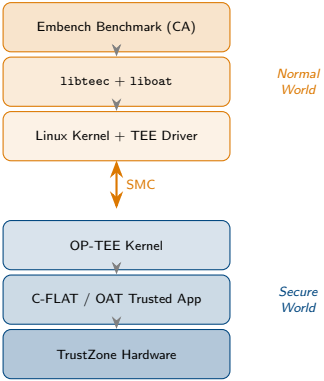
Background: OP-TEE

OP-TEE — Open Portable Trusted Execution Env.

OP-TEE is an **open-source TEE** from Linaro that runs in ARM TrustZone's Secure World.

- Implements the **GlobalPlatform TEE API**
- Hosts small programs called **Trusted Applications (TAs)**
- Normal World talks to it via `libteec` + SMC
- Provides secure storage, crypto primitives, and attestation logic

In our work: The C-FLAT Trusted Application (TA) lives inside OP-TEE. It receives control-flow hashes from the instrumented benchmark and validates them against a *reference CFG*.

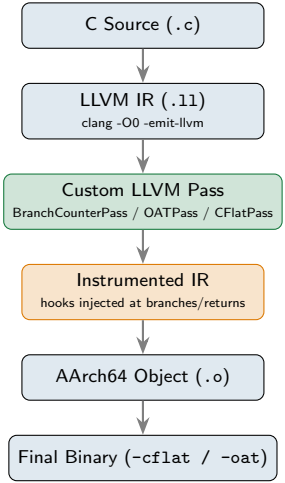


Background: LLVM & Compiler Instrumentation

Why LLVM?

LLVM is a modern compiler framework with a **pass plugin** system. We write custom *LLVM passes* that automatically insert attestation hooks into any C program at compile time.

- **No source changes needed** — works on unmodified benchmark C files
- Operates on **LLVM IR** (intermediate representation) — architecture-neutral
- Cross-compile the instrumented IR for **AArch64** (ARM) with a single flag



Threat Model

[THREAT] Attacker Capabilities

- Full control of **Normal World** (Linux + apps)
- Can modify flight-controller firmware or inject malicious waypoints
- Can launch **code-reuse attacks** (ROP/JOP) in Normal World
- **Cannot** compromise TrustZone's Secure World
- **Cannot** tamper with hardware root-of-trust

[SEC] What We Protect

- **Control-flow integrity** of the flight-control program
- **Attestation evidence** (stored in Secure World, signed with TEE keys)
- **Reference CFG** (Control-Flow Graph) of the expected execution path

[WARN] Primary Attack Scenario

An adversary injects a payload that alters the UAV's flight path (via GPS spoofing or memory corruption). The CFA scheme must **detect the deviation** and report it to the ground station.

What Is Control-Flow Integrity (CFI) Verification?

The Key Insight

Every program follows a finite set of valid **control-flow paths**. An attack *must* deviate from these paths. So verifying the path $\equiv$ verifying integrity.

How CFA Works

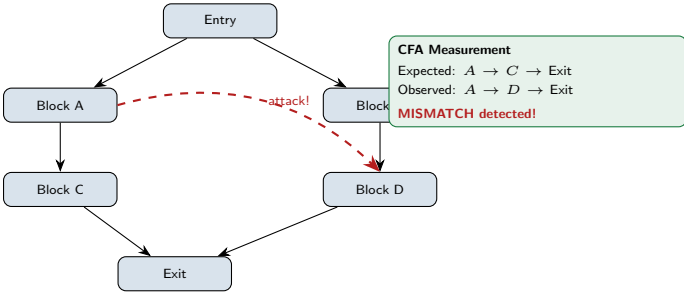
At build time: extract the valid Control-Flow Graph (CFG).

At runtime: record executed path as a measurement.

At verification: compare measurement to the CFG.

Mismatch $\Rightarrow$ Attack detected.

Control-Flow Graph (CFG)



How C-FLAT Counts World Switches

C-FLAT: Every Control-Flow Event

C-FLAT instruments **every** control-flow transfer so the TEE can reconstruct the full execution path hash. Each instrumented event triggers one SMC call (world switch):

Event type	Switches
Conditional branch (taken/not-taken)	N_{cond}
Unconditional branch	N_{uncond}
Loop back-edge (per iteration)	N_{loop}
Direct call	N_{dcall}
Indirect call / jump	N_{icall}
Return	N_{ret}
Init + Finalize (fixed)	$+2$

$$S_{\text{C-FLAT}} = N_{cond} + N_{uncond} + N_{loop} + N_{dcall} + N_{icall} + N_{ret} + 2$$

Why So Many Switches?

C-FLAT aims for **full path coverage** — the verifier must see every branch outcome to reconstruct the exact execution trace and detect any deviation.

Each SMC call transfers the branch ID and outcome into the TEE, where the running path hash is updated:

$$H_{\text{new}} = \text{Hash}(H_{\text{prev}} \| \text{BranchID})$$

Measured C-FLAT Counts (AArch64)

Benchmark	Total switches
aha-mont64	857.8 M
crc32	871.9 M
primecount	1,607.2 M
sglib-combined	1,461.7 M
nbody	17.3 M
cubic	2.0 M

How OAT Counts World Switches

OAT: Minimum Switch Set $S_{\min}$

OAT (S&P 2019) defines a **minimal** set of events that still provides attestation security, avoiding unnecessary TEE transitions. Only *unpredictable* targets need a switch:

Event type	OAT	C-FLAT
Conditional branch	✓	✓
Indirect call/jump	✓	✓
Return	✓	✓
Direct call	✗	✓
Unconditional branch	✗	✓
Loop back-edge	✗	✓
Init + Finalize	$+2$	$+2$

$$S_{\min} = N_{cond} + N_{icall} + N_{ret} + 2$$

Why Exclude Direct Calls & Unconditionals?

Direct calls: target is *statically known* at compile time — an attacker cannot redirect them without modifying code, which is detected by memory integrity checks.

Unconditional branches & loop back-edges: these carry no branching decision; including them adds overhead without improving attack detection coverage.

OAT Reference Counts (Paper Table II)

Benchmark	$S_{\min}$ switches
aha-mont64	392,967,008
crc32	348,840,008
primecount	880,206,008
sglib-combined	757,568,416
nbody	6,329,070
cubic	860,013

OAT switches ≈ 45–55% of C-FLAT — significant saving.

Why Do We Need Performance Benchmarking?

UAV Real-Time Constraints

UAV flight-control loops run at **fixed frequencies**. Attestation that breaks these deadlines is **a safety problem, not just a performance problem.**

Attestation must not:

- Delay motor commands past the control loop period
- Cause jitter in sensor fusion pipelines
- Block GPS or IMU data processing

Benchmarking lets us ask:

1. Which code regions fit within the real-time deadline?
2. Does OAT or C-FLAT suit the UAV timing budget?
3. Where to use lightweight counting vs. full TEE?

Embench-IOT: Why This Suite?

Built around **diverse control-flow**

- Rich mix of branches, loops, and calls
- Mimics UAV workloads: tight loops and conditionals
- Lets us measure OAT vs. C-FLAT overhead on *realistic* paths
- Avoids flat, straight-line code that would undercount real-world switches

Security vs. Real-Time: The Core Tradeoff

More instrumentation $\Rightarrow$ stronger attestation, but more control-loop interruptions. Benchmarking **quantifies** this tradeoff for principled deployment decisions.

Implementation Overview

	Branch Counter	OAT	C-FLAT Count-Only
Source	`embench-iot-benchmarks/`	`embench-iot-oat-count/`	`embench-iot-cflat-count/`
Patch	`patch.sh`	`patch_oat.sh`	`patch_cflat.sh`
Build	`build_optee.sh`	`build_oat_count.sh`	`min_cflat.sh`
LLVM Pass	`BranchCounterPass.so`	`OATPass.so`	`CFlatPass.so`
Output	`output_optee/`	`output_oat_countonly/`	`output_cflat_countonly/`

All pipelines: **clang -O0 -emit-llvm** → strip `optnone` → **opt** (pass plugin) → **aarch64-linux-gnu-gcc** (cross-link)

The LLVM Instrumentation Pipeline (Step by Step)

Pipeline for every benchmark:

1. **Generate LLVM IR** at -O0:

```
clang -O0 -S -emit-llvm \
  --target=aarch64-linux-gnu \
  mont64.c -o mont64.ll
```

2. **Strip** optnone so passes run:

```
opt -strip-debug -S mont64.ll | \
  sed 's/optnone/noinline/g' \
  > mont64_stripped.ll
```

3. **Run pass plugin:**

```
opt -load-pass-plugin=OATPass.so \
    -passes="oat-pass" \
    -S mont64.ll -o mont64_inst.ll
```

4. **Compile + cross-link:**

```
clang -c mont64_inst.ll -o mont64.o
aarch64-linux-gnu-gcc \
  mont64.o liboat_count_only.o \
  -o mont64-oat
```

What each pass injects:

BranchCounterPass

`increment_branch(id)` at every branch edge. Counts branch frequency — lightweight baseline.

OATPass (Smin)

`__oat_log(val)` at cond. branch targets,
`__oat_log_indirect(addr)` at indirect calls,
`__oat_func_exit(id)` at returns.
Direct calls excluded per OAT paper.

CFlatPass

`cflat_record(id)` at cond. branches, uncond. branches, loop headers, calls, returns. Full path coverage for count-only mode.

Patching the Benchmark Source Files

`patch_*.sh` scripts use `sed` to inject runtime hooks into Embench C sources.

`patch.sh` (Branch Counter)

```
// After warm_caches():
init_branch_stats();

// Before verify_benchmark():
print_branch_stats();
```

Backup created before any patch

`patch_oat.sh` (OAT)

```
// Before benchmark():
__oat_init();

// Before verify_benchmark():
__oat_print_proof();
```

warm_caches() **excluded** from OAT scope.

Idempotent — skips already-patched files

`patch_cflat.sh` (C-FLAT)

```
// Before verify_benchmark():
cflat_finalize_and_print();
```

`min_libcflat` counts locally, no TEE dependency.

Pattern-based via `grep` on Embench signatures

Measured Results: Branch Counter Statistics (1/2)

Benchmark	Cond. Br.	Uncond. Br.	Loop Hdrs	Direct Calls	Returns
aha-mont64	384.5M	456.4M	189.9M	8.5M	8.5M
crc32	174.4M	348.7M	174.3M	174.4M	174.4M
cubic	0.7M	1.0M	0.3M	2.0M	0.2M
edn	371.9M	732.8M	360.9M	0.7M	0.7M
huffbench	495.8M	486.3M	233.3M	1.1M	1.1M
matmult-int	406.7M	794.1M	387.4M	0.1M	0.1M
minver	109.3M	156.0M	56.6M	6.1M	6.1M
nbody	6.2M	10.8M	4.6M	1.1M	0.1M
nettle-aes	78.0M	147.7M	51.2M	0.9M	0.9M
nettle-sha256	30.4M	185.3M	24.2M	2.4M	3.8M
primecount	880.2M	727.0M	282.3M	<0.1M	<0.1M
sglib-combined	680.9M	627.5M	147.8M	76.6M	76.6M
st	9.2M	18.3M	9.1M	8.0M	7.9M
tarfind	80.9M	114.0M	48.4M	36.3M	36.3M
ud	412.4M	616.3M	255.7M	1.5M	1.5M

All counts in millions (M) · CPU_MHZ=1000 · AArch64 OP-TEE build

Measured Results: CFLAT & OAT World Switch Counts (2/2)

Benchmark	CFLAT Switches	OAT Switches	Ratio (CFLAT/OAT)
aha-mont64	857.8M	466.6M	×1.84
crc32	871.9M	348.8M	×2.50
cubic	2.0M	0.9M	×2.36
edn	1,106.1M	372.6M	×2.97
huffbench	984.2M	602.6M	×1.63
matmult-int	1,201.0M	406.8M	×2.95
minver	277.5M	131.0M	×2.12
nbody	17.3M	6.3M	×2.73
nettle-aes	227.4M	86.8M	×2.62
nettle-sha256	221.8M	36.1M	×6.14
primecount	1,607.2M	880.2M	×1.83
sglib-combined	1,461.7M	990.6M	×1.48
st	43.3M	17.1M	×2.53
tarfind	267.6M	142.7M	×1.88
ud	1,031.6M	437.5M	×2.36

All counts in millions (M) · CFLAT: cond.+uncond. branches, loop headers, calls, returns · OAT: cond. branches, indirect calls, returns (Smin)

Conclusion

We built a complete, reproducible CFA benchmarking framework for embedded UAV workloads.

What we built:

- 3 LLVM pass plugins: Branch Counter, OAT, and C-FLAT (count-only) instrumentation
- 4 end-to-end build pipelines (host + AArch64 cross)
- 3 patch scripts for automatic runtime hook injection
- Complete switch-count data for 15 Embench benchmarks
- `min_libcflat` runtime: TEE-free C-FLAT count without any libteec dependency

What we showed:

- World-switch counts span **6 orders of magnitude** across benchmarks
- C-FLAT count-only and OAT count-only are comparably lightweight and deployable on bare AArch64
- Selective attestation of safety-critical UAV subsystems is the recommended deployment strategy

Thank You

Presentation By - Aastha Jain[JCS242802]

Guided By - Prof. Vireshwar Kumar

[GITHUB] `https://github.com/Aastha-jain06/Embench`